Функции

Вы уже знаете, что каждая программа должна содержать функцию main() (с которой она и начинает свое выполнение). Так же не менее, большинство программ использует и много других функций.

Оглавление:

1. Функции
2. Возвращаемые значения
3. Тип возврата void
4. Возврат значений функций main()
5. Еще о возвращаемых значениях
6. Повторное использование функций
7. Вложенные функции
8. Тест

Функции

Функция — это последовательность, отвечающая для выполнения определенного задания. Часто ваша программа будет прерывать выполнение одной функций ради выполнения другой. Вы делаете аналогичные вещи в реальной жизни постоянно. Например, вы на месте книги и вспоминаете, что должны были сделать телефонный звонок. Вы отложили закладку в какой книги, берете телефон и набираете номер. После того, как вы уже поговорили, вы возвращаетесь к отложе с той страницы, на которой остановились.

Программы на языке C++ работают похожим образом. Иногда, когда программа выполняет код, она может столкнуться с ситуацией, которая требует, чтобы программа выполнила какое-то другое задание. Программа запоминает, где она остановилась и отправляется выполнять другой фрагмент. Процессор не помнит закладку и текущий точка выполнения, а затем выполняет вызываемый фрагмент. Когда выполнение вызываемой функции завершено, процессор возвращается к закладке и возобновляет выполнение прерванной функции.

Функция, в которой находится вызов, называется **caller**, а функция, которая вызывает — **вызываемая функция**, например:

```
// Function prototypes
void getHello();
int getNumber();
int add(int, int);

int main()
{
    int sum = 0;
    int x;

    // Call getHello() function twice
    getHello();
    getHello();

    // Call getNumber() function twice
    x = getNumber();
    x = getNumber();

    // Call add() function with the values 1 and 2
    sum = add(1, 2);

    // Call add() function with the values 3 and 4
    sum = add(3, 4);

    // Call add() function with the values x and 5
    sum = add(x, 5);

    // Call add() function with the values sum and 6
    sum = add(sum, 6);

    return 0;
}

void getHello()
{
    // Print "Hello" to the console
    cout << "Hello";
}

int getNumber()
{
    // Ask the user for a number
    cout << "Enter an integer: ";
    int x;
    cin >> x;

    return x;
}

int add(int a, int b)
{
    int c = a + b;

    return c;
}
```

Результат выполнения программы:

```
Enter an integer: 4
Enter an integer: 5
```

Сначала функция main() начинает выполнение. Затем, когда программа выполняет код, она может столкнуться с ситуацией, которая требует, чтобы программа выполнила какое-то другое задание. Программа запоминает, где она остановилась и отправляется выполнять другой фрагмент. Процессор не помнит закладку и текущий точка выполнения, а затем выполняет вызываемый фрагмент. Когда выполнение вызываемой функции завершено, процессор возвращается к закладке и возобновляет выполнение прерванной функции.

Функция, в которой находится вызов, называется **caller**, а функция, которая вызывает — **вызываемая функция**, например:

```
// Function prototypes
void getHello();
int getNumber();
int add(int, int);

int main()
{
    int sum = 0;
    int x;

    // Call getHello() function twice
    getHello();
    getHello();

    // Call getNumber() function twice
    x = getNumber();
    x = getNumber();

    // Call add() function with the values 1 and 2
    sum = add(1, 2);

    // Call add() function with the values 3 and 4
    sum = add(3, 4);

    // Call add() function with the values x and 5
    sum = add(x, 5);

    // Call add() function with the values sum and 6
    sum = add(sum, 6);

    return 0;
}

void getHello()
{
    // Print "Hello" to the console
    cout << "Hello";
}

int getNumber()
{
    // Ask the user for a number
    cout << "Enter an integer: ";
    int x;
    cin >> x;

    return x;
}

int add(int a, int b)
{
    int c = a + b;

    return c;
}
```

Результат выполнения программы:

```
Hello
Hello
Enter an integer: 4
Enter an integer: 5
4 + 5 = 12
```

Здесь main() прерывается 2 раза. Обратите внимание, в обоих случаях, получение пользовательского значения сохраняется в переменной x, а затем передается обратно в main() с помощью return, где присваивается переменной x или 0.

Также main() не является единственной функцией, которая может вызывать другие функции. Любая функция может вызывать другие функции:

```
// Function prototypes
void print0();
void print1(int);

int main()
{
    print0();
    print1(5);

    return 0;
}

void print0()
{
    // Print "0" to the console
    cout << "0";
}

void print1(int x)
{
    // Print the value that was passed in to this function
    cout << x;
}
```

Результат выполнения программы:

```
05
```

Разберемся детально:

- Первый вызов функции main() вызывает print0(). В обратном в caller, которая затем передается в std::cout для вывода.
- Второй вызов функции main() опять вызывает print0(). В обратном в caller. Выражение 5 + 3 имеет значение 8, который затем выводится на экран.
- Третий вызов функции main() опять вызывает print0(). В обратном в caller. Однако функция main() начнет с нуля не делает, поэтому начнет и не прерывает (возвращающее значение игнорируется).

Примечание: Возвращаемые значения не выводятся на экран, если он не передать объекту std::cout. В последнем вызове функции print0() значение не отправляется в std::cout, поэтому ничего и не происходит.

Тип возврата void

Функции могут и не возвращать значения. Чтобы сообщить компилятору, что функция не возвращает значение, нужно использовать **тип возврата void**. Включая еще раз на функции getHello() из вышеприведенного примера:

```
void getHello()
{
    // Print "Hello" to the console
    cout << "Hello";
}
```

Эта функция имеет тип возврата void, который означает, что функция не возвращает значения. Поскольку значение не возвращается, то и операция return не требуется.

Все еще один пример использования функции void:

```
// Function prototype
void printSomething();

int main()
{
    int sum = 0;

    // Call printSomething() function
    printSomething();

    // Call printSomething() function again
    printSomething();

    return 0;
}

void printSomething()
{
    // Print "Hello" to the console
    cout << "Hello";
}
```

В первом вызове функции return0Nothing() выводится "0", но ничего не возвращается обратно в caller. Точка выполнения возвращается обратно в функцию main(), где программа продолжает свое выполнение.

Второй вызов функции return0Nothing() даже не скомпилируется. Функция return0Nothing() имеет тип возврата void, который означает, что эта функция не возвращает значения. Однако функция main() пытается отправить это значение которое не возвращается в std::cout для вывода. Std::cout не может обработать этот случай, так как значение на выходе не предоставлено. Следовательно, компилятор выдаст ошибку. Или лучше будет рассмотреть случай, чтобы компилятор пошел дальше.

Возврат значений функцией main()

Теперь у вас есть понимание того, как работают функции void. Когда программа выполняется, операционная система делает вызов функции main() и начинается её выполнение. Стабильность в main() выполняется последовательно. В конце функции main() возвращает целочисленное значение обратно. В обратном в операционную систему. Поэтому main() объявляется как int main().

Почему нужно возвращать значение обратно в операционную систему? Дело в том, что возвращение значения функции main() является **кодом состояния**, который сообщает операционной системе об успешном или неудачном выполнении программы. Обычно возвращение значения 0 (ноль) означает что все функции успешно, тогда как любое другое значение означает неудачу/ошибку.

Обратите внимание, по стандартам языка C++ функции main() должна возвращать целочисленное значение. Однако, если вы не указали этого и хотя бы функция main(), компилятор возвратит 0 автоматически, если нельзя ошибок не будет. Но рекомендуется указывать явно в конце функции main() и использовать тип возврата int для функции main().

Еще о возвращаемых значениях

Во первых, тип возврата функции не обязательно должен быть целочисленным. Функции могут возвращать почти любой тип(int). Единственное исключение — функции main(), которая возвращает 0, если не предусмотрено другое значение.

Во вторых, когда процессор встречает в функции оператор return, он немедленно выполняет возврат значения обратно в caller и точка выполнения также переходит в caller. Любой код, который находится за оператором return в функции — игнорируется.

Функция может возвращать только одно значение через return обратно в caller. Это может быть либо число (например, 5), либо значение переменной, либо выражение (в которого есть результат), либо определенное значение из набора возможных значений.

Но есть способы обойти правило возврата одного значения, возвращая сразу несколько значений, но об этом детальнее мы поговорим на соответствующем уроке.

Наконец, автор функции решает, что означает не возвращающее значение. Некоторые функции используют возвращаемые значения как код ошибки для указания результата выполнения функции (успешно не выполнено или нет). Другие функции возвращают определенное значение из набора возможных значений. Кроме того, существует функции, которые вообще ничего не возвращают.

Повторное использование функций

Одна и та же функция можно вызывать несколько раз, даже в разных программах, что очень полезно:

```
// Function prototype
int getNumber();

int main()
{
    int sum = 0;

    // Call getNumber() function twice
    sum = getNumber();
    sum = getNumber();

    return 0;
}

int getNumber()
{
    // Ask the user for a number
    cout << "Enter an integer: ";
    int x;
    cin >> x;

    return x;
}
```

Результат выполнения программы:

```
Enter an integer: 4
Enter an integer: 5
4 + 5 = 12
```

Здесь main() прерывается 2 раза. Обратите внимание, в обоих случаях, получение пользовательского значения сохраняется в переменной x, а затем передается обратно в main() с помощью return, где присваивается переменной x или 0.

Также main() не является единственной функцией, которая может вызывать другие функции. Любая функция может вызывать другие функции:

```
// Function prototypes
void print0();
void print1(int);

int main()
{
    print0();
    print1(5);

    return 0;
}

void print0()
{
    // Print "0" to the console
    cout << "0";
}

void print1(int x)
{
    // Print the value that was passed in to this function
    cout << x;
}
```

Результат выполнения программы:

```
05
```

Разберемся детально:

- Первый вызов функции main() вызывает print0(). В обратном в caller, которая затем передается в std::cout для вывода.
- Второй вызов функции main() опять вызывает print0(). В обратном в caller. Выражение 5 + 3 имеет значение 8, который затем выводится на экран.
- Третий вызов функции main() опять вызывает print0(). В обратном в caller. Однако функция main() начнет с нуля не делает, поэтому начнет и не прерывает (возвращающее значение игнорируется).

Примечание: Возвращаемые значения не выводятся на экран, если он не передать объекту std::cout. В последнем вызове функции print0() значение не отправляется в std::cout, поэтому ничего и не происходит.

Тип возврата void

Функции могут и не возвращать значения. Чтобы сообщить компилятору, что функция не возвращает значение, нужно использовать **тип возврата void**. Включая еще раз на функции getHello() из вышеприведенного примера:

```
void getHello()
{
    // Print "Hello" to the console
    cout << "Hello";
}
```

Эта функция имеет тип возврата void, который означает, что функция не возвращает значения. Поскольку значение не возвращается, то и операция return не требуется.

Все еще один пример использования функции void:

```
// Function prototype
void printSomething();

int main()
{
    int sum = 0;

    // Call printSomething() function
    printSomething();

    // Call printSomething() function again
    printSomething();

    return 0;
}

void printSomething()
{
    // Print "Hello" to the console
    cout << "Hello";
}
```

В первом вызове функции return0Nothing() выводится "0", но ничего не возвращается обратно в caller. Точка выполнения возвращается обратно в функцию main(), где программа продолжает свое выполнение.

Второй вызов функции return0Nothing() даже не скомпилируется. Функция return0Nothing() имеет тип возврата void, который означает, что эта функция не возвращает значения. Однако функция main() пытается отправить это значение которое не возвращается в std::cout для вывода. Std::cout не может обработать этот случай, так как значение на выходе не предоставлено. Следовательно, компилятор выдаст ошибку. Или лучше будет рассмотреть случай, чтобы компилятор пошел дальше.